Connection to the Problem Statement:

The main problems that the project addresses are that the University of Evansville Swimming and Diving team currently manages class schedules, team schedules, meet results, and other information across multiple systems. The University of Evansville also does not provide a protected period of time for athletes to practice. This makes coaches need to coordinate times that work best for practice. Coaches must manually compare swimmers' class schedules when deciding practice times.

My proposed system addresses these problems by combining scheduling and swimming performance information into one platform for coaches and swimmers. The scheduling side can currently identify conflicts between proposed practice times and athletes' class schedules. It can also recommend practice times with the fewest conflicts while considering several preferences selected by the coach. The performance side gives coaches and swimmers one place to view results, personal bests, and progression. The main intended users are the University of Evansville Swimming & Diving coaching staff and swimmers.

System Design / Architecture:

The project uses a React/Vite frontend and a Python/FastAPI backend.

The frontend has two separate views for the coach and swimmer within the current prototype. The frontend is responsible for displaying schedules, calendars, performance information, and scheduling controls.

The backend does all of the scheduling calculations and work. When a coach checks a practice time or searches for the best practice times, the frontend sends the necessary schedule information and settings to the backend. The backend uses the conflict-detection and practice-recommendation logic and sends the results back to the frontend.

The current prototype uses fictional data for schedules and results. Later, I plan to add a relational database as a major improvement to the project. This will help store and retrieve information instead of what is currently done in the existing prototype.

User Requirements:

Coaches need a system to provide an organized way to view swimmer schedules, identify scheduling conflicts, compare possible practice times, and view swimmer performance information. Swimmers need to be able to view their own schedule, team events, personal results, and performance progression without having access to information that should only be available to coaches. The project will reduce the amount of time that coaches spend finding practice times significantly while still allowing the coach to make the final decision on time. The system is designed to assist the coaching staff rather than replace their decision-making.

Functional Requirements:

1. The system must allow coaches to check proposed practice times against swimmer schedules and identify which swimmers have conflicts.
2. The system must generate possible practice times based on a selected time range, practice duration, and transition time, then rank the options by the number of conflicts.
3. The system must provide Coach and Swimmer views that display information appropriate for each type of user.
4. The system must display swimmer performance information including personal bests, recent results, improvement, and progression over time.
5. The system must provide a calendar that displays team events and swimmer class schedules.

Non-Functional Requirements:

1. **Usability and accessibility**: The interface should be easy to understand and easy to use for coaches and swimmers. Important information such as scheduling conflicts should be given through feedback using text and labels rather than relying solely on color, for example.
2. **Privacy and security**: Athlete class schedules and performance information should only be available to appropriate users. The current prototype uses fictional data while privacy and future role-based access are being developed. This is more of a future requirement, but it is extremely important to consider during development.
3. **Performance and reliability**: Scheduling calculations should return fast enough to be useful during normal team planning, and the system should consistently identify conflicts when the entered schedule information is correct. The system should work accordingly if everything is entered correctly and much faster than what is done currently.

Implementation and Validation Plan:

The project is being developed using React, Vite, JavaScript, and CSS for the frontend and Python, FastAPI, and Uvicorn for the backend. The entire development of the project is managed through Git and GitHub. The current system uses fictional data to avoid privacy and security concerns; a relational database will be implemented later to house this information with a persistent storage method.

Implementation is being completed incrementally to ensure that individual features are added and work correctly. The scheduling system currently includes automated Python unit tests for conflict detection, transition-time handling, practice-time ranking, recommendation generation, and API behavior. The frontend is also checked using its production build command and through manual testing of the main Coach and Swimmer workflows.

Validation includes testing various scheduling situations where conflicts are expected to appear and checking to ensure that recommended practice windows are ranked correctly and verifying that performance calculations display the expected results. Later testing will also include feedback from clients and users to evaluate usability and whether the system actually makes scheduling and accessing team information easier.